JobImpact AI | P59

Job Task Exposure Analyzer

Track E: Responsible AI

Anurag Chotulal Dube · Roll 59

MIT ADT University · B.Tech Computer Science Engineering

Objective

Break a job into responsibilities, assess task exposure, identify related skills and outline reskilling steps. This is educational decision support, not an employment prediction tool.

Problem statement

AI affects tasks differently. Routine repeatable work may be automated; some work is augmented by tools; contextual judgment, empathy, accountability, and physical work often remain human-led.

Scope

The project implements task decomposition, exposure classification, semantic skill matching, proficiency self-assessment, roadmap generation, dynamic infographic generation, and controlled responsible-AI probes.

Tools and model status

Technology

Python, Streamlit, Gemini API (optional), Sentence-BERT all-MiniLM-L6-v2, PyTorch runtime, Pandas, NumPy, scikit-learn, Plotly, Matplotlib and Pillow.

Data

The evaluation CSV contains 19 manually labelled educational examples across Software Developer, Data Analyst and Digital Marketing Specialist. It is a small classroom reference set, not occupational ground truth.

Model status

No model fine-tuning is performed or claimed. Without Gemini credentials, clearly labelled illustrative fallbacks are used. Sentence-BERT may fall back to TF-IDF if the model cannot load.

System architecture



Implementation and responsible use

U1–U2

Prompt comparison shows zero-shot versus few-shot decomposition. Classification uses task-level categories: Automate, Augment, and Human-led, with explanations. Prompting is inference; fine-tuning would need training data and validation.

U3–U4

Sentence embeddings match responsibilities to a structured skill vocabulary. Similarity is semantic proximity, not a probability or proficiency. Self-ratings inform priorities; a staged plan and image are generated from analysis results.

U5

Controlled demographic wording probes detect changes in model output on a tiny set; they cannot certify fairness. Structural checks confirm allowed labels and explanations, not factual correctness.

Privacy and oversight

Descriptions are not stored in an application database. With Gemini configured, entered text is sent to Google's API. Avoid personal or confidential details. A human must review all outputs; do not use the system for employment decisions.

Limitations

Role descriptions omit context; skills list is incomplete; models can hallucinate; API model availability changes; similarity and exposure are not calibrated. The system does not forecast layoffs, adoption, or wages.

Executed evaluation results

Run mode

Rule-based illustrative demo fallback (not Gemini). Executed against the manually labelled educational set.

Dataset and accuracy

19 examples; 0.9474 (94.74%) accuracy on this small educational dataset.

Mean latency

0.000011 seconds per row (local demo loop; excludes UI and network). Not a Gemini inference latency.

Confusion matrix

Rows: reference labels; columns: predictions; order: Automate, Augment, Human-led

[[6 0 0]
 [0 6 0]
 [1 0 6]]

Failure examples

Software Developer: Generate boilerplate for a security-sensitive authentication flow — expected Human-led, got Automate.

Interpretation

This measures only the simple demo rules against hand-labelled examples developed with the project, not Gemini or Sentence-BERT performance. Results are not statistically representative and should not be generalized.

Conclusion and demonstration evidence

Conclusion

JobImpact AI demonstrates an end-to-end, task-level educational workflow from role input to exposure analysis, skill gaps, and reskilling support. Keeping assumptions visible and requiring human review are core design decisions.

Screenshots

No screenshots are included because a full Streamlit runtime was unavailable during this build session. Capture genuine screenshots after installing dependencies and running the application.

Repository

Git repository initialized locally. GitHub URL and live deployment: not created or verified.

Next steps

Install dependencies in Python 3.11, configure Gemini secret if desired, run every page, capture genuine screenshots, execute both live and fallback evaluations, and update this report with final runtime/model details.